Computability – Topic 10

Yonghao Lee

March 3, 2026

Contents

1 Additional NP-Complete Problems **2**
1.1 The Vertex Cover Problem . . . 3
1.2 The Dominating Set Problem . . . 4
1.3 CNF Formulas and k-SAT . . . 6
1.4 k-SAT $\leq_P$ 3-SAT . . . 6

1 Additional NP-Complete Problems

The phenomenon of NP-completeness is widespread. NP-complete problems appear in many fields across computer science, mathematics, and operations research. Interestingly, most naturally occurring NP-problems are known either to be in P or to be NP-complete; very few problems have an intermediate status. If you seek a polynomial-time algorithm for a new NP-problem, spending part of your effort attempting to prove it NP-complete is sensible, because doing so may prevent you from searching for a polynomial-time algorithm that does not exist.

We now present additional theorems showing that various languages are NP-complete. These theorems provide examples of the techniques used in proofs of this kind. Our general strategy is to exhibit a polynomial-time reduction from 3SAT to the language in question, though we sometimes reduce from other languages already known to be NP-complete when that is more convenient.

Remark 1.1: Why reduce from 3SAT?

We typically reduce from 3SAT rather than SAT because the rigid structure of 3cnf-formulas (exactly three literals per clause) makes it easier to design gadgets. However, once we have established the NP-completeness of additional problems (such as CLIQUE or VERTEX-COVER), we may reduce from whichever NP-complete problem has the most convenient structure for the target problem at hand.

When constructing a polynomial-time reduction from 3SAT to a language, we look for structures in that language that can simulate the variables and clauses in Boolean formulas. Such structures are sometimes called **gadgets**. For example, in the reduction from 3SAT to CLIQUE, the triples of nodes serve as clause gadgets (one triple per clause), and the edge-removal rules serve as consistency gadgets (preventing contradictory variable assignments).

Theorem 1.1: CLIQUE is NP-Complete

CLIQUE is NP-complete.

Proof. We verify both conditions of NP-completeness.

Condition 1: CLIQUE $\in$ NP. Given an instance $\langle G, k\rangle$, a certificate is a set S of k vertices. A verifier checks that every pair of vertices in S is connected by an edge in G. This requires checking at most $\binom{k}{2}$ pairs, which is polynomial in the size of the input. Therefore CLIQUE $\in$ NP.

Condition 2: CLIQUE is NP-hard. We have already shown 3SAT $\leq_P$ CLIQUE in Topic 9 (see the Cook–Levin theorem and subsequent reductions). Since 3SAT is NP-complete, by the transferability of NP-completeness, CLIQUE is NP-complete. $\square$

Now we use the fact that CLIQUE is NP-complete to show the completeness of other languages involving graphs.

Definition 1.1: Independent Set

An **independent set** of a graph $G = (V, E)$ is a subset $I \subseteq V$ such that no two vertices in I are adjacent—that is, for every $u, v \in I$, the edge $\{u, v\} \notin E$. The language:

$$\text{IS} = \{\langle G, k\rangle \mid G \text{ has an independent set of size } k\}$$

Remark 1.2: Duality of Cliques and Independent Sets

A clique and an independent set are dual notions. A set $C \subseteq V$ is a clique in $G = (V, E)$ if and only if C is an independent set in the **complement graph** $\overline{G} = (V, \overline{E})$, where $\overline{E} = \{\{u, v\} \mid u \neq v \text{ and } \{u, v\} \notin E\}$. This duality is the key insight behind the following reduction.

Theorem 1.2: IS is NP-Complete

IS is NP-complete.

Proof. **Condition 1: IS $\in$ NP.** Given an instance $\langle G, k\rangle$, a certificate is a set I of k vertices. A verifier checks that no two vertices in I are connected by an edge in G, which requires checking at most $\binom{k}{2}$ pairs. Therefore IS $\in$ NP.

Condition 2: IS is NP-hard. We show CLIQUE $\leq_P$ IS.

Construction. Given an instance $\langle G, k\rangle$ of CLIQUE, the reduction outputs $\langle \overline{G}, k\rangle$, where $\overline{G}$ is the complement graph of G.

Correctness. We show $\langle G, k\rangle \in$ CLIQUE $\iff$ $\langle \overline{G}, k\rangle \in$ IS.

Let $C \subseteq V$. Then C is a clique in G (every pair in C is connected in G) if and only if C is an independent set in $\overline{G}$ (no pair in C is connected in $\overline{G}$, since the edges are exactly the non-edges of G). In particular, G has a clique of size k if and only if $\overline{G}$ has an independent set of size k.

Running time. Constructing $\overline{G}$ requires copying the vertex set V and, for every pair $u, v \in V$, including $\{u, v\}$ in $\overline{E}$ if and only if $\{u, v\} \notin E$. This takes $O(|V|^2)$ time, which is polynomial. Therefore CLIQUE $\leq_P$ IS, and since CLIQUE is NP-complete, IS is NP-complete. $\square$

1.1 The Vertex Cover Problem

Definition 1.2: Vertex Cover

If $G = (V, E)$ is an undirected graph, a **vertex cover** of G is a subset $C \subseteq V$ such that every edge of G is incident to at least one vertex in C. That is, for every $\{u, v\} \in E$, at least one of u or v belongs to C. The language:

$$\text{VC} = \{\langle G, k\rangle \mid G \text{ has a vertex cover of size at most } k\}$$

Remark 1.3: Duality of Independent Sets and Vertex Covers

Independent sets and vertex covers are complementary structures. A set $I \subseteq V$ is an independent set in $G = (V, E)$ if and only if $V \setminus I$ is a vertex cover of G. To see why: I is an independent set means no edge has both endpoints in I, which means every edge has at least one endpoint in $V \setminus I$, which is exactly the definition of $V \setminus I$ being a vertex cover. This complementarity is the key insight behind the following reduction.

Theorem 1.3: VC is NP-Complete

VC is NP-complete.

Proof. **Condition 1: VC $\in$ NP.** Given an instance $\langle G, k\rangle$, a certificate is a set C of at most k vertices. A verifier checks that for every edge $\{u, v\} \in E$, at least one of u, v belongs to C. This requires checking at most $|E|$ edges, which is polynomial in the input size. Therefore VC $\in$ NP.

Condition 2: VC is NP-hard. We show IS $\leq_P$ VC.

Construction. Given an instance $\langle G, k\rangle$ of IS where $G = (V, E)$, the reduction outputs $\langle G, |V| - k\rangle$. Note that the graph G itself is unchanged; only the numerical parameter changes.

Correctness. We show $\langle G, k\rangle \in$ IS $\iff$ $\langle G, |V| - k\rangle \in$ VC.

($\Rightarrow$) Suppose G has an independent set I of size k. Let $C = V \setminus I$, so $|C| = |V| - k$. We claim C is a vertex cover. Consider any edge $\{u, v\} \in E$. Since I is an independent set, u and v cannot both be in I. Therefore at least one of them is in $V \setminus I = C$, so C covers this edge. Since this holds for every edge, C is a vertex cover of size $|V| - k$.

($\Leftarrow$) Suppose G has a vertex cover C of size at most $|V| - k$. Let $I = V \setminus C$, so $|I| \geq k$. We claim I is an independent set. Consider any two vertices $u, v \in I$. Since neither u nor v is in C, and C is a vertex cover (every edge must touch C), the edge $\{u, v\}$ cannot exist in E. Therefore no two vertices in I are adjacent, so I is an independent set of size at least k.

Running time. The reduction simply outputs the same graph G together with the number $|V| - k$. Computing $|V| - k$ takes constant time. Therefore the reduction runs in polynomial time, so IS $\leq_P$ VC, and since IS is NP-complete, VC is NP-complete. □

We now use the NP-completeness of Vertex Cover to establish the NP-completeness of a related graph problem: the Dominating Set problem. While vertex cover asks us to cover *edges*, dominating set asks us to cover *vertices* — a seemingly different requirement that turns out to be at least as hard.

1.2 The Dominating Set Problem

Given an undirected graph $G = \langle V, E \rangle$, a **dominating set** in G is a set $D \subseteq V$ such that for every $v \in V$, either $v \in D$ or there exists $u \in D$ such that $\{u, v\} \in E$; that is, every vertex is at distance at most 1 from some vertex in D.

Definition 1.3: Dominating Set

$$DS = \{\langle G, k \rangle \mid G \text{ has a dominating set of size } k\}$$

Theorem 1.4: DS is NP-Complete

DS is NP-complete.

Proof. **DS $\in$ NP:** We describe a polynomial-time verifier for DS. The certificate is a subset $D \subseteq V$ with $|D| = k$. Given D, the verifier checks that for every vertex $v \in V$, either $v \in D$ or there exists some $u \in D$ such that $\{u, v\} \in E$. This can be done in polynomial time by iterating over all vertices and checking adjacency.

VC $\leq_P$ DS**:**

High-level idea: Vertex Cover and Dominating Set are fundamentally different: VC asks us to *cover edges* (every edge must have an endpoint in the set), while DS asks us to *dominate vertices* (every vertex must be in the set or adjacent to it). The challenge is to translate edge-covering constraints into vertex-domination constraints. The key trick is: for each edge $e = \{u, v\}$, we create a new vertex w_e whose *only* neighbors are u and v. Now the only way to dominate w_e is to place u or v into the dominating set — which is exactly the same constraint as covering the edge $\{u, v\}$ in vertex cover. This creates a one-to-one correspondence between "covering edge e" and "dominating vertex w_e."

Construction: Given an instance $\langle G = \langle V, E \rangle, k \rangle$ of Vertex Cover, we construct an instance $\langle G', k' \rangle$ of Dominating Set as follows.

Let $I = \{v \in V \mid \deg(v) = 0\}$ be the set of isolated vertices in G (vertices not incident to any edge), and let $i = |I|$.

Why do we care about isolated vertices? An isolated vertex has no neighbors, so in G' it will still have no neighbors (we only add edges to new vertices w_e, which connect to endpoints of edges). Therefore, the only way to dominate an isolated vertex is to put it directly into the dominating set. These vertices play no role in vertex cover (they touch no edges), so we must "pay" for them separately.

Construct $G' = \langle V', E' \rangle$ where:

- For every edge $e = \{u, v\} \in E$, introduce a new vertex w_e.
- $V' = V \cup \{w_e \mid e \in E\}$
- $E' = E \cup \{\{u, w_e\}, \{v, w_e\} \mid e = \{u, v\} \in E\}$

Set $k' = k + i$.

Runtime: This construction runs in polynomial time: we add $|E|$ new vertices and $2|E|$ new edges, and computing i takes at most $O(|V| + |E|)$ time.

Correctness ($\Rightarrow$): Suppose G has a vertex cover S of size k. We claim that $D = S \cup I$ is a dominating set in G' of size $k + i = k'$.

Intuition: the vertex cover already "covers" every edge, meaning for each edge at least one endpoint is in S. In G', each new vertex w_e is adjacent only to the endpoints of e, so it is automatically dominated. The original non-isolated vertices are dominated because the vertex cover forces a neighbor into S. The isolated vertices are handled by including them directly.

We verify that every vertex in V' is dominated by D:

- *Isolated vertices:* Every $v \in I$ is in D by construction, so it is trivially dominated.
- *Original non-isolated vertices:* Let $v \in V \setminus I$. Then v is incident to some edge $\{u, v\} \in E$. Since S is a vertex cover, either $v \in S \subseteq D$ (dominated trivially) or $u \in S \subseteq D$ (and v is adjacent to $u \in D$ via the edge $\{u, v\} \in E \subseteq E'$).
- *New vertices w_e:* For each w_e where $e = \{u, v\} \in E$, since S covers e, at least one of u, v is in $S \subseteq D$, and that vertex is adjacent to w_e by construction of G'.

Thus every vertex in V' is dominated, so D is a dominating set of size $k' = k + i$.

Correctness ($\Leftarrow$): Suppose G' has a dominating set D' of size at most k'. We want to show that G has a vertex cover of size at most k. We do this in three steps: remove isolated vertices, swap out new vertices, then show what remains is a vertex cover.

Intuition: the dominating set D' might contain "junk" — isolated vertices and new vertices w_e — that do not exist in G or are irrelevant to vertex cover. We clean up D' step by step until we are left with a set of original, non-isolated vertices. Then, because each w_e still needs to be dominated and its only neighbors are the endpoints of edge e, the cleaned-up set must cover every edge.

Step 1: Isolated vertices must be in D'. Every isolated vertex $v \in I$ has no neighbors in G, and we did not add any new edges to it in G', so it still has no neighbors in G'. The only way to dominate v is if $v \in D'$. Hence $I \subseteq D'$. These i vertices are useless for covering edges (they touch no edges), so we set them aside. Let $D = D' \setminus I$. Then $|D| \leq k' - i = k$.

Step 2: Swap out any new vertex w_e from D. The set D might still contain new vertices w_e that do not exist in G. We fix this: if D contains some w_e for $e = \{u, v\}$, replace w_e with u. This is safe because:

- w_e is adjacent only to u and v.
- u is adjacent to v, to w_e, and to every $w_{e'}$ for edges e' incident to u.

So u dominates everything w_e did and more. The size does not increase. After all such replacements, we obtain a set $D \subseteq V$ of size at most k that is still a dominating set of G'.

Step 3: Show D is a vertex cover of G. Pick any edge $e = \{u, v\} \in E$. The vertex w_e still exists in G' (we removed it from the *set*, not from the *graph*). Since D is a dominating set, w_e must be dominated by some vertex in D. But $D \subseteq V$, and the only vertices in V adjacent to w_e are u and v. So $u \in D$ or $v \in D$, which means edge e is covered.

This holds for every edge, so D is a vertex cover of G with $|D| \leq k$. $\square$

Having established the NP-completeness of several graph problems, we now turn to a different kind of reduction: showing relationships between variants of the satisfiability problem itself. Specifically, we show that the general k-SAT problem (where clauses can have k literals) is no harder than 3-SAT.

1.3 CNF Formulas and k-SAT

Recall that a Boolean formula is called k-CNF if it is in conjunctive normal form and each clause has exactly k literals. The corresponding decision problem is:

Definition 1.4: k-SAT

$$k\text{-SAT} = \{\langle\varphi\rangle \mid \varphi \text{ is a satisfiable } k\text{-CNF formula}\}$$

We already know that 3-SAT is NP-complete (from the Cook–Levin theorem and subsequent reductions). We now show that k-SAT for any k is no harder than 3-SAT, by reducing k-SAT to 3-SAT.

1.4 k-SAT $\leq_P$ 3-SAT

Theorem 1.5: k-SAT reduces to 3-SAT

k-SAT $\leq_P$ 3-SAT.

High-level idea: We want to show that if we could solve 3-SAT efficiently, we could solve k-SAT efficiently too. The idea is to take any clause with more than 3 literals and break it into smaller clauses of size 3, using fresh auxiliary variables. Clauses with fewer than 3 literals are padded by repeating a literal. The trick is to do this in a way that preserves satisfiability — the new formula is satisfiable if and only if the original one is.

Handling short clauses ($k \leq 3$):

If a clause already has 3 literals, we leave it unchanged. If a clause has fewer than 3 literals, we pad it with duplicate literals to reach size 3:

- A clause $(\ell_1 \vee \ell_2)$ of size 2 becomes $(\ell_1 \vee \ell_2 \vee \ell_2)$.
- A clause (ℓ_1) of size 1 becomes $(\ell_1 \vee \ell_1 \vee \ell_1)$.

This is valid because $\ell \vee \ell$ is logically equivalent to ℓ, so the padded clause is satisfied by exactly the same assignments as the original. No new variables are needed.

How to shorten a single clause by one ($k > 3$):

Let $x_1, \ldots, x_n$ be variables, and let $\phi = (\ell_1 \vee \ell_2 \vee \cdots \vee \ell_k)$ be a clause of $k > 3$ literals. The idea is to introduce a new variable y that "acts as" $\ell_1 \vee \ell_2$. Consider the formula:

$$\phi' = (y \vee \ell_3 \vee \ell_4 \vee \cdots \vee \ell_k) \wedge (\bar{\ell_1} \vee y \vee y) \wedge (\bar{\ell_2} \vee y \vee y) \wedge (\ell_1 \vee \ell_2 \vee \bar{y})$$

What is going on here? We replaced the first two literals ℓ_1, ℓ_2 in the original clause with a single new variable y. The three extra clauses enforce that y behaves exactly like $\ell_1 \vee \ell_2$:

- $(\bar{\ell_1} \vee y \vee y)$: if ℓ_1 is true, then y must be true. (i.e., $\ell_1 \Rightarrow y$.)
- $(\bar{\ell_2} \vee y \vee y)$: if ℓ_2 is true, then y must be true. (i.e., $\ell_2 \Rightarrow y$.)
- $(\ell_1 \vee \ell_2 \vee \bar{y})$: if y is true, then at least one of ℓ_1, ℓ_2 must be true. (i.e., $y \Rightarrow \ell_1 \vee \ell_2$.)

Together, these three clauses are logically equivalent to $y \Leftrightarrow (\ell_1 \vee \ell_2)$. The first clause $(y \vee \ell_3 \vee \cdots \vee \ell_k)$ is the original clause with $\ell_1 \vee \ell_2$ replaced by y. Since y faithfully represents $\ell_1 \vee \ell_2$, the new formula ϕ' is satisfiable if and only if the original clause ϕ is.

Notice that the main clause shrank from k literals to $k-1$ literals, and the three auxiliary clauses each have exactly 3 literals. So we reduced the clause size by one.

Key correctness lemma:

Lemma 1.1: Clause Shortening Preserves Satisfiability

Every assignment $A : \{x_1, \ldots, x_n\} \to \{\text{TRUE}, \text{FALSE}\}$ that satisfies ϕ can be extended to an assignment $A' : \{x_1, \ldots, x_n, y\} \to \{\text{TRUE}, \text{FALSE}\}$ that satisfies ϕ'. Conversely, every assignment $A' : \{x_1, \ldots, x_n, y\} \to \{\text{TRUE}, \text{FALSE}\}$ that satisfies ϕ' also satisfies ϕ when restricted to $\{x_1, \ldots, x_n\}$.

Proof. **($\Rightarrow$):** Let A be an assignment that satisfies ϕ. Extend it to A' by setting $A'(y) = A(\ell_1 \vee \ell_2)$, i.e., y gets the truth value of $\ell_1 \vee \ell_2$ under A. Since the three auxiliary clauses are equivalent to $y \Leftrightarrow (\ell_1 \vee \ell_2)$, A' satisfies them by construction. Since A' agrees with A on all original variables, and A satisfies $(\ell_1 \vee \ell_2 \vee \cdots \vee \ell_k)$, A' satisfies $(y \vee \ell_3 \vee \cdots \vee \ell_k)$ as well: if one of $\ell_3, \ldots, \ell_k$ was true under A, it is still true; if not, then $\ell_1 \vee \ell_2$ was true, so y is true. Hence A' satisfies ϕ'.

($\Leftarrow$): Let A' be an assignment that satisfies ϕ'. The auxiliary clauses force y to have the same truth value as $\ell_1 \vee \ell_2$. Since A' satisfies $(y \vee \ell_3 \vee \cdots \vee \ell_k)$, either y is true (meaning $\ell_1 \vee \ell_2$ is true) or some ℓ_j for $j \geq 3$ is true. Either way, the original clause $(\ell_1 \vee \ell_2 \vee \cdots \vee \ell_k)$ is satisfied. So restricting A' to $\{x_1, \ldots, x_n\}$ gives a satisfying assignment for ϕ. □

Proof of the theorem:

Proof. **(I) Construction:** Given a k-CNF formula φ, we construct a 3-CNF formula $\psi = f(\varphi)$ as follows. Go through each clause of φ:

- If the clause has fewer than 3 literals, pad it with duplicate literals to reach size 3.
- If the clause has exactly 3 literals, leave it unchanged.
- If the clause has $k > 3$ literals, repeatedly apply the shortening procedure: introduce a fresh variable y, replace the clause with the four new clauses described above. The main clause now has $k - 1$ literals. If $k - 1 > 3$, repeat with another fresh variable. Continue until the main clause has exactly 3 literals.

For a single clause of size $k > 3$, we apply the procedure $k - 3$ times, introducing $k - 3$ new variables. Each application produces 3 auxiliary clauses of size 3 and shortens the main clause by one literal.

(II) Correctness:

($\Rightarrow$): If $\langle\varphi\rangle \in k$-SAT, then there is a satisfying assignment to φ. By the lemma, we can extend it to a satisfying assignment to ψ by choosing each new variable y according to $y \Leftrightarrow (\ell_1 \vee \ell_2)$, as in the construction. Each application of the shortening procedure preserves satisfiability, so the entire chain does too. Hence $\langle\psi\rangle \in$ 3-SAT.

($\Leftarrow$): If $\langle\psi\rangle \in$ 3-SAT, then there is a satisfying assignment to ψ. By the lemma, each application of the shortening procedure preserves satisfiability in both directions. So if we restrict the assignment to the original variables in φ, it satisfies φ. Hence $\langle\varphi\rangle \in k$-SAT.

(III) Complexity: The procedure of shortening a single clause takes polynomial time: we search for a clause longer than 3, create a new variable name, and add the new clauses. For a clause of size k, we repeat $k - 3$ times. Padding short clauses takes constant time per clause. The number of clauses in φ is bounded by the input size, and the number of repetitions per clause is bounded by k (which is at most the input size). Therefore f can be computed in polynomial time. □